

Első lépések C++-ban: kiírás, változók, bekérés

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Programozni úgy lehet megtanulni, ahogy biciklizni: nem olvasással, hanem kipróbálással. Ebben a jegyzetben minden újdonságot azonnal ki is próbálsz.

- **Próbáld ki** – gépeld be a kódot, és futtasd. Gépeld, ne másold: a kezded is tanul.
- **Tippelj** – előbb írd le a füzetedbe, szerinted mit ír ki a program, és csak utána futtasd. Ha tévedtél, annál jobb: most tanultál valamit.
- **Kísérlet** – változtass meg valamit, és figyeld meg, mi történik. Nem tudsz semmit elrontani: ha a program hibás lesz, kijavítod, és újra futtatod.
- **Feladat** – ezt már te írod meg. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb. Minden feladatnál ott van, mit kell kapnod, így magad ellenőrizheted a munkádat.

Két tudnivaló, mielőtt belevágsz:

- A kiírt szövegekbe egyelőre ne írd ékezetes betűt, mert a program ablaka nem mindig jeleníti meg jól.
- Ha hibaüzenetet kapsz, ne ijedj meg. A jegyzet végén a Hibakereső táblázat megmondja, mit jelent, és mit tegyél.

1. Az első programod

A program utasítások sorozata, amelyeket a számítógép sorban végrehajt. Az első programod egyetlen dolgot csinál: kiír egy mondatot a képernyőre.

Próbáld ki

```
#include <iostream>

int main() {
    std::cout << "Hello, vilag!" << std::endl;
    return 0;
}
```

Gépeld be pontosan így, aztán fordítsd le és futtasd, ahogy a tanárod mutatta. Ha megjelent a **Hello, vilag!** felirat, megírtad életed első programját.

Nézzük meg, melyik sor mit jelent:

- **#include <iostream>** – ezzel kérjük el a kiíráshoz és a beolvasáshoz szükséges eszközöket.
- **int main() {** – itt kezdődik a program. A kapcsos zárójel nyitja meg a program törzsét.
- **std::cout << "Hello, vilag!"** – a **std::cout** a képernyőre ír. A **<<** jel azt mutatja, merre megy az adat: a szöveg a képernyő felé.
- **"Hello, vilag!"** – a kiírandó szöveg. A szöveget mindig idézőjelek közé írjuk.
- **<< std::endl** – a sor vége: a következő kiírás új sorban kezdődik.
- **;** – minden utasítást pontosvessző zár le, ahogy a mondatot a pont.
- **return 0;** – jelzi, hogy a program rendben véget ért.
- **}** – a program törzsének vége.

Kísérlet - írd át!

Cseréld le a **Hello, világ!** szöveget valami másra, és futtasd újra. Aztán próbáld ki, mi történik, ha a szövegbe számokat vagy írásjeleket írsz.

Próbáld ki - több sor

A program törzsébe több utasítást is írhatsz. A gép sorban hajtja végre őket:

```
#include <iostream>

int main() {
    std::cout << "Első sor" << std::endl;
    std::cout << "Második sor" << std::endl;
    std::cout << "Harmadik sor" << std::endl;
    return 0;
}
```

Tippelj - mi történik a sorvége nélkül?

```
std::cout << "alma";
std::cout << "fa";
std::cout << std::endl;
std::cout << "korte" << "fa" << std::endl;
```

Hány sor jelenik meg, és mi áll bennük?

Amit megfigyelhettél: új sor csak ott kezdődik, ahol **std::endl** áll. Egy utasításban több dolgot is kiírhatsz, mindegyik elé **<<** jel kerül.

Feladatok

1.1. feladat ★☆☆ - Bemutatkozás

Írj programot, amely kiírja a nevedet.

Ellenőrzés: egyetlen sor jelenik meg, benne a neved.

1.2. feladat ★☆☆ - Három sor

Írd ki három külön sorba a nevedet, az osztályodat és a kedvenc ételedet.

Ellenőrzés: három sor, például **Kiss Anna, 9.B, rantott sajt.**

1.3. feladat ★☆☆ - Két utasítás, egy sor

Írd ki ezt a mondatot: **Jo reggelt, Anna!** – de úgy, hogy a köszönés és a név két külön **std::cout** utasításban legyen.

Ellenőrzés: a mondat egyetlen sorban jelenik meg. Figyelj a vessző utáni szóközre!

1.4. feladat ★☆☆ - Üres sor

Írj ki két mondatot úgy, hogy közöttük egy üres sor legyen.

Ellenőrzés: három sor jelenik meg, a középső üres.

1.5. feladat ★★☆☆ - Fenyőfa

Rajzolj fenyőfát csillagokból. A szóközők is számítanak!

Ellenőrzés:

```
  *
 ***
*****
*****
  |
```

1.6. feladat ★★☆☆ - Névjegykártya

Készíts névjegykártyát kerettel, a saját adataiddal.

Ellenőrzés: valami ilyesmi jelenik meg, és a keret jobb széle egy vonalban van:

```
+-----+
| Kiss Anna   |
| 9.B osztaly |
+-----+
```

2. A program szerkezete

Minden C++ program ugyanarra a vázra épül. A te dolgod a váz belsejét megírni.

A váz

```
#include <iostream>

int main() {
    // ide jonnek az utasitasaid
    return 0;
}
```

Ezt a hat sort minden programod elejére és végére leírod. Az utasításaid mindig a `{` és a `return 0;` közé kerülnek.

A `//` jellel kezdődő rész **megjegyzés**. A számítógép nem foglalkozik vele: neked és a programod olvasójának szól.

Négy szabály, amelyet érdemes megjegyezni:

- A gép az utasításokat felülről lefelé, sorban hajtja végre.
- Minden utasítás végén pontosvessző áll.
- A kis- és nagybetű különbözik: a `main` nem ugyanaz, mint a `Main`.
- Az utasításokat négy szóközzel beljebb kezdjük. A gépnek ez mindegy, de így sokkal könnyebb olvasni a programot.

Tippelj - milyen sorrendben?

```
std::cout << "harom" << std::endl;
std::cout << "egy" << std::endl;
std::cout << "ketto" << std::endl;
```

Kísérlet - rontsuk el szándékosan!

A programozók naponta kapnak hibaüzenetet; ez a munka része. A hibaüzenet nem büntetés, hanem segítség: megmondja, hol keresd a hibát. Most szándékosan elrontjuk az első programot, hogy megismerd a leggyakoribb üzeneteket.

Végezd el az alábbi változtatásokat **egyesével**. Mindegyik után próbáld lefordítani a programot, olvasd el a hibaüzenetet, keresd meg a Hibakeresőben, aztán javítsd vissza a programot.

1. Töröld ki az egyik pontosvesszőt.
2. Töröld ki a szöveg végéről az idézőjelet.
3. A `std::cout` helyett írd csak ezt: `cout`.
4. A `std::cout` helyett írd ezt: `std::cuot`.
5. Töröld ki a program utolsó sorából a `}` jelet.

Tipp: A hibaüzenetben szerepel egy szám: annak a sornak a sorszáma, ahol a fordító elakadt. A hiba gyakran az **előző** sorban van (például ott maradt le a pontosvessző). Ha több hibaüzenetet kapsz, mindig a legelső javítsd ki először, aztán fordíts újra.

Feladatok

2.1. feladat ★☆☆ - Hibavadászat

Ebben a programban négy hiba van. Gépeld be pontosan így, aztán keresd meg és javítsd ki mindet.

```
#include <iostream>

int main() {
    std::cout << "Szia! << std::endl;
    std::cout << "Hibat keresek." << std::endl
    cout << "Megtalaltam!" << std::endl;
    return 0;
```

Ellenőrzés: a javított program három sort ír ki: `Szia!`, `Hibat keresek.`, `Megtalaltam!`

2.2. feladat ★☆☆ - Megjegyzések

Írj az előző programod elejére megjegyzést: a nevedet és azt, hogy mit csinál a program. Aztán írd `//` jelet a középső `std::cout` sor elé, és futtasd. Mi változott?

Ellenőrzés: a megjegyzéssel „kikapcsolt” sor nem jelenik meg; a program két sort ír ki.

2.3. feladat ★☆☆ - Recept

Írj programot, amely egy egyszerű recept (például szendvics vagy tea) lépéseit írja ki sorszámozva. Utána cseréld fel a programban két utasítás sorrendjét, és futtasd újra.

Ellenőrzés: a lépések pontosan abban a sorrendben jelennek meg, ahogy az utasítások a programban állnak.

3. Számolás

A program számolni is tud. Amit idézőjelbe teszel, azt betűről betűre kiírja; amit nem, azt kiszámolja.

Tippelj

```
std::cout << "3 + 4" << std::endl;
std::cout << 3 + 4 << std::endl;
```

Próbáld ki - szöveg és eredmény együtt

```
std::cout << "3 + 4 = " << 3 + 4 << std::endl;
std::cout << "Egy napban " << 24 * 60 << " perc van." << std::endl;
```

Tippelj - számológép

A műveleti jelek: + összeadás, - kivonás, * szorzás, / osztás, % maradék. Töltsd ki a középső oszlopot fejből, aztán írasd ki mindegyiket a programmal.

Kifejezés	A tippem	A program szerint
$2 + 3 * 4$		
$(2 + 3) * 4$		
$10 - 4 - 3$		
$7 / 2$		
$7.0 / 2$		
$7 \% 2$		
$20 / 3$		
$20 \% 3$		

Volt meglepetés? Három dolgot érdemes megjegyezni:

- A műveletek sorrendje ugyanaz, mint matematikából: előbb a szorzás és az osztás, utána az összeadás és a kivonás. Zárójellel ezt megváltoztathatod.
- Ha két **egész** számot osztasz, az eredmény is egész: a tizedesek elvesznek. Ha az egyik szám tizedes tört (7.0), az eredmény is az lesz. A tizedes törtekben pontot írunk, nem vesszőt.
- A $\%$ az osztás **maradékát** adja meg: 20-ban a 3 megvan 6-szor, és marad 2.

Feladatok

3.1. feladat ★☆☆ - Számológép

Számold ki a programmal: $125 + 378$, $1000 - 387$, $24 * 15$.

Ellenőrzés: 503, 613, 360.

3.2. feladat ★☆☆ - Szépen kiírva

Írd ki az előző három műveletet úgy, hogy a művelet is látszódjon: $125 + 378 = 503$. Az eredményt a program számolja ki, ne te írd be!

Ellenőrzés: három sor, a fenti formában.

3.3. feladat ★☆☆ - Egy hét

Hány óra, hány perc és hány másodperc van egy hétben?

Ellenőrzés: 168 óra, 10080 perc, 604800 másodperc.

3.4. feladat ★☆☆ - Átlag

Három jegyed van: 4, 5 és 4. Számold ki az átlagukat. Először írd így: $(4 + 5 + 4) / 3$. Jó az eredmény? Javítsd ki!

Ellenőrzés: a helyes átlag 4.33333.

3.5. feladat ★★☆☆ - Cukorka

17 cukorkát osztasz szét 5 gyerek között egyenlően. Számold ki, hány cukorkát kap egy gyerek, és hány marad ki.

Ellenőrzés: mindenki 3 cukorkát kap, és 2 marad.

4. Változók

A változó egy névvel ellátott tároló. Megjegyezz egy értéket, amelyet később felhasználhatsz és megváltoztathatsz.

Próbáld ki

```
#include <iostream>

int main() {
    int kor = 16;

    std::cout << "Eletkorom: " << kor << std::endl;
    return 0;
}
```

Az `int kor = 16;` sor három részből áll:

- `int` – a változó **típusa**: egész számot tárol.
- `kor` – a változó **neve**. Ezt te választod.
- `= 16` – a kezdő **értéke**.

Képzeld el egy dobozt, amelyre ráírtad: `kor`. A dobozban most a 16-os szám van.

Tippelj – idézőjellel vagy anélkül?

```
int kor = 16;

std::cout << "kor" << std::endl;
std::cout << kor << std::endl;
```

Próbáld ki – számolás változókkal

```
int szelesseg = 5;
int magassag = 3;
int terület = szelesseg * magassag;

std::cout << "Terület: " << terület << std::endl;
```

Írd át a szélességet 12-re, a magasságot 7-re, és futtasd újra. A program újraszámolta a területet: ez a változók igazi haszna.

Tippelj – az érték megváltozik

```
int x = 10;
std::cout << x << std::endl;

x = 20;
std::cout << x << std::endl;

x = x + 5;
std::cout << x << std::endl;
```

A programozásban az `=` jel nem azt jelenti, hogy „egyenlő”. Azt jelenti: **számold ki a jobb oldalt, és tedd be a bal oldalon álló változóba**. Ezért értelmes az `x = x + 5;` utasítás: vedd az `x` mostani értékét, adj hozzá ötöt, és az eredményt tedd vissza az `x`-be. A változó régi értéke ilyenkor elvész.

Milyen típusok vannak?

Típus	Mit tárol?	Példa
<code>int</code>	egész szám	<code>int kor = 16;</code>
<code>double</code>	tizedes tört	<code>double magassag = 1.72;</code>
<code>std::string</code>	szöveg	<code>std::string nev = "Anna";</code>
<code>char</code>	egyetlen karakter	<code>char betu = 'A';</code>
<code>bool</code>	igaz vagy hamis	<code>bool esik = true;</code>

A `std::string` használatához a program elejére kell egy újabb sor: `#include <string>`

Kísérlet - mi történik, ha...?

Próbáld ki ezeket egyesével, és figyeld meg az eredményt vagy a hibaüzenetet:

1. Egész típusú változóba tizedes törtet teszel: `int ar = 3.99;`, aztán kiírod.
2. Nem adsz kezdőértéket: `int y;`, aztán kiírod az `y` értékét.
3. Olyan változót írsz ki, amelyet nem hoztál létre: `std::cout << z << std::endl;`

Figyelem! Egy változót használat előtt létre kell hozni, és értéket kell adni neki. Érték nélkül értelmetlen szám lehet benne.

A változók nevére vonatkozó szabályok:

- Betűkből, számjegyekből és aláhúzásjelből állhat, de nem kezdődhet számjeggyel.
- Nem lehet benne szóköz és ékezetes betű.
- Legyen beszédes: a `kor` és a `terulet` jó név, az `a` és az `x2` nem árul el semmit.

Feladatok

4.1. feladat ★☆☆ - Adatlap

Hozz létre három változót: `nev` (szöveg), `kor` (egész) és `magassag` (tizedes tört, méterben). Adj nekik értéket, és írd ki őket három sorba.

Ellenőrzés: például `Nev: Anna, Kor: 16, Magassag: 1.72`

4.2. feladat ★☆☆ - Téglalap

Bővítsd a területszámoló programot: számolja ki a téglalap területét is.

Ellenőrzés: 5 és 3 esetén a terület 15, a kerület 16. Ha a két értéket 12-re és 7-re írod át: 84 és 38.

4.3. feladat ★☆☆ - Kövesd papíron

Ezt a feladatot előbb papíron oldd meg. Írd be a táblázatba, mennyi az **a** és a **b** értéke az egyes sorok végrehajtása után. Az első két sort kitöltöttem.

```
int a = 5;
int b = 3;
a = a + b;
b = a - b;
a = a * 2;
```

A végrehajtott sor	a értéke	b értéke
int a = 5;	5	még nincs
int b = 3;	5	3
a = a + b;		
b = a - b;		
a = a * 2;		

Ellenőrzés: gépeld be a programot, és a végén írasd ki a két változót. Ezt kell kapnod: **a** értéke 16, **b** értéke 5.

4.4. feladat ★☆☆ - Bolt

Egy füzet ára 350 Ft, és 4 darabot veszel. Tárold az árat és a darabszámot egy-egy változóban, számold ki a fizetendő összeget, és írd ki így: **4 db, darabja 350 Ft, összesen 1400 Ft**

Ellenőrzés: a fenti sor. Írd át a darabszámot 7-re: az összeg 2450 lesz.

4.5. feladat ★☆☆ - Csere

Legyen **a** értéke 3, **b** értéke 8. Cseréld meg a két változó értékét úgy, hogy a program végén **a** legyen 8, **b** pedig 3. A 3-as és a 8-as számot nem írhatod le újra!

Először próbáld ki ezt a két sort, és írasd ki az eredményt: **a = b;** majd **b = a;**. Mi romlott el, és miért?

Tipp: kell egy harmadik változó.

Ellenőrzés: a javított program végén **a** értéke 8, **b** értéke 3.

5. Bekérés

Eddig az adatokat beleírtuk a programba. A **std::cin** segítségével a program futás közben kér adatot a felhasználótól, így minden futtatáskor mással dolgozhat.

Próbáld ki

```
#include <iostream>

int main() {
    int kor;

    std::cout << "Hany éves vagy? ";
    std::cin >> kor;

    std::cout << "Jovore " << kor + 1 << " éves leszel." << std::endl;
    return 0;
}
```


Futtasd, írd be az életkorodat, és nyomj Entert. Futtasd újra egy másik számmal.

- `int kor;` – létrehozuk a változót. Kezdőértéket most nem adunk neki, mert az értéke a billentyűzetről érkezik.
- `std::cout << "Hany eves vagy? ";` – előbb megkérdezzük, mit várunk. Itt nincs `std::endl`, ezért a válasz a kérdés mellé kerül.
- `std::cin >> kor;` – a program megáll, és megvárja, amíg beírsz egy számot és Entert nyomsz. A beírt szám a `kor` változóba kerül.

Figyeld meg a nyilak irányát! A `std::cout <<` a képernyő felé küldi az adatot, a `std::cin >>` pedig a változó felé.

Kísérlet - kérdés nélkül

Töröld ki (vagy `//` jellel kapcsold ki) a `Hany eves vagy?` kérdést kiíró sort, és futtasd a programot. Mit lát a felhasználó? Honnan tudná, mit kell tennie?

Figyelem! Bekérés előtt mindig írd ki, mit vársz a felhasználtól. Kérdés nélkül a program szó nélkül várakozik, és úgy tűnik, mintha lefagyott volna.

Próbáld ki - szöveg bekérése

```
#include <iostream>
#include <string>

int main() {
    std::string nev;

    std::cout << "Mi a keresztned? ";
    std::cin >> nev;

    std::cout << "Szia, " << nev << "!" << std::endl;
    return 0;
}
```

A `std::cin >>` egyelőre egyetlen szót tud beolvasni. Próbáld ki, mi történik, ha a teljes nevedet írod be.

Próbáld ki - több adat

```
int a, b;

std::cout << "Elso szam: ";
std::cin >> a;
std::cout << "Masodik szam: ";
std::cin >> b;

std::cout << "Az osszeguk: " << a + b << std::endl;
```

Az `int a, b;` sorral egyszerre két változót hoztunk létre.

Kísérlet - rossz adat

Futtasd az életkoros programot, és szám helyett írd be ezt: `tizenhat`. Mit ír ki a program?

A program azt várja, amit a változó típusa megenged. Ha mást kap, nem jelez hibát, csak értelmetlen eredményt ad. Egyelőre abból indulunk ki, hogy a felhasználó azt írja be, amit kérünk.

Feladatok

5.1. feladat ★☆☆ - Köszönés

Kérd be a felhasználó keresztnévét, és köszönj neki név szerint.

Ellenőrzés: ha a beírt név **Anna**, a kimenet **Szia, Anna! Örülök, hogy megismertelek.**

5.2. feladat ★☆☆ - Életkor

Kérd be a felhasználó életkorát. Írd ki, hány éves lesz jövőre, és hány éves lesz tíz év múlva.

Ellenőrzés: 16 esetén **Jövőre 17 éves leszel.** és **Tíz év múlva 26 éves leszel.**

5.3. feladat ★☆☆ - Két szám

Kérj be két egész számot. Írd ki az összegüket, a különbségüket és a szorzatukat.

Ellenőrzés: 12 és 5 esetén 17, 7 és 60.

5.4. feladat ★☆☆ - Téglalap bekéréssel

Kérd be egy téglalap szélességét és magasságát, és írd ki a területét és a kerületét.

Ellenőrzés: 5 és 3 esetén a terület 15, a kerület 16.

5.5. feladat ★☆☆ - Átlag

Kérj be két egész számot, és írd ki az átlagukat. Vigyázz az egész osztásra!

Ellenőrzés: 3 és 4 esetén 3.5; 10 és 20 esetén 15.

5.6. feladat ★☆☆ - Vásárlás

Kérd be egy termék nevét (egy szó), az árát és azt, hány darabot vesz belőle a vásárló. Írd ki, mennyit kell fizetni.

Ellenőrzés: **fuzet**, 350 és 4 esetén: **4 db fuzet ara 1400 Ft.**

5.7. feladat ★☆☆ - Hőmérséklet

Kérj be egy hőmérsékletet Celsius-fokban (tizedes tört is lehet), és írd ki Fahrenheit-fokban. A képlet: $F = C \cdot 9 / 5 + 32$.

Ellenőrzés: 20 → 68; 37 → 98.6; 0 → 32.

5.8. feladat ★☆☆ - Percek és másodpercek

Kérj be egy időtartamot másodpercben, és írd ki percben és másodpercben. **Tipp:** az osztás és a maradék segít.

Ellenőrzés: 200 → **3 perc 20 masodperc**; 59 → **0 perc 59 masodperc**.

5.9. feladat ★★ - Pénzváltó

Kérj be egy százzal osztható összeget. Írd ki, hogyan fizethető ki a lehető legkevesebb ezressel, ötszázassal és százassal.

Ellenőrzés: 3700 → 3 db 1000-es, 1 db 500-as, 2 db 100-as. 900 → 0 db 1000-es, 1 db 500-as, 4 db 100-as.

6. Egy teljes program

A legtöbb program három lépésből áll: adatot kér, számol, majd kiírja az eredményt.

Próbáld ki

```
#include <iostream>
#include <string>

int main() {
    // 1. bemenet
    std::string uticel;
    int letszam;
    int buszdi;

    std::cout << "Hova megy az osztaly? ";
    std::cin >> uticel;
    std::cout << "Hanyan mennek? ";
    std::cin >> letszam;
    std::cout << "Mennyibe kerul a busz? ";
    std::cin >> buszdi;

    // 2. szamolas
    int fejenkent = buszdi / letszam;

    // 3. kimenet
    std::cout << "Uticel: " << uticel << std::endl;
    std::cout << "Egy fore " << fejenkent << " Ft jut." << std::endl;
    return 0;
}
```

Próbáld ki ezekkel az adatokkal: **Eger**, 30 fő, 90000 Ft. Egy főre 3000 Ft jut.

Most próbáld ki 28 fővel. Mennyit ír ki a program? Ha mindenki ennyit fizet, összejön a busz ára? Számold ki a programmal, mennyi hiányzik!

Feladatok

6.1. feladat ★★☆☆ - Belépőjegy

Bővítsd a kirándulás programot: kérje be a fejenkénti belépőjegy árát is, és írja ki, mennyibe kerül a kirándulás egy főnek összesen.

Ellenőrzés: **Eger**, 30 fő, 90000 Ft busz, 1500 Ft belépő → fejenként 4500 Ft.

6.2. feladat ★★☆☆ - Hány éves?

Kérd be a felhasználó születési évét és az idei évet. Írd ki, hány éves lesz az idei év végén.

Ellenőrzés: 2010 és 2026 esetén 16.

6.3. feladat ★★☆☆ - Pizza

Egy pizza 8 szeletből áll. Kérd be, hány pizzát rendeltek, és hányan eszik. Írd ki, hány szelet jut egy főre, és hány szelet marad.

Ellenőrzés: 3 pizza és 5 fő esetén mindenkinek 4 szelet jut, és 4 szelet marad.

6.4. feladat ★★☆☆ - Zsebpénz

Kérd be, mennyi zsebpénzt kap valaki hetente, és mennyibe kerül, amire gyűjt. Írd ki, hány hét alatt gyűlik össze a pénz. Vigyázz: ha az osztás nem jön ki pontosan, akkor egy héttel több kell!

Ellenőrzés: heti 1500 Ft és 20000 Ft-os cél esetén 14 hét (13 hét alatt csak 19500 Ft gyűlik össze). Heti 2000 Ft és 20000 Ft esetén pontosan 10 hét.

6.5. feladat ★★★ - A saját programod

Találj ki egy saját programot! Kérjen be legalább két adatot, számoljon velük valamit, és írja ki az eredményt érthető mondatban. Mutasd meg a padtársadnak: ki tudja próbálni magyarázat nélkül?

Ellenőrzés: a program akkor jó, ha a padtársad a kiírt kérdésekből megérti, mit kell beírnia.

Hibakereső: mit jelent a hibaüzenet?

Keresd meg a táblázatban azt a sort, amely hasonlít arra, amit látsz. A hibaüzenet pontos szövege fordítónként kicsit eltérhet.

Amit látsz	Mit jelent?	Mit tegyél?
<code>expected ';' before ...</code>	Lemaradt egy pontosvessző.	Nézd meg a jelzett sor előtti sor végét.
<code>missing terminating " character</code>	Lemaradt egy idézőjel a szöveg végéről.	Minden szöveg elején és végén legyen idézőjel.
<code>'cout' was not declared in this scope</code>	A fordító nem ismeri ezt a szót.	Írd elé: <code>std::cout</code> . Megvan az <code>#include <iostream></code> sor?
<code>'cout' is not a member of 'std'</code>	Elírtál egy szót.	Ellenőrizd betűről betűre: <code>cout</code> , <code>cin</code> , <code>endl</code> .
<code>expected '}' at end of input</code>	Lemaradt a záró kapcsos zárójel.	A program utolsó sora <code>}</code> legyen.
<code>'kor' was not declared in this scope</code>	Olyan változót használsz, amelyet nem hoztál létre.	Létrehoztad (<code>int kor;</code>)? Ugyanúgy írtad a nevét? Ha szöveg, van körülötte idézőjel?
<code>'string' is not a member of 'std'</code>	Hiányzik a szövegkezelés fejléce.	<code>#include <string></code>
<code>undefined reference to 'main' vagy 'WinMain'</code>	Nincs <code>main</code> a programban.	Elírtad a <code>main</code> szót?
Nagyon hosszú hiba, benne <code>no match for 'operator>>'</code> vagy <code>'operator<<'</code>	Rossz irányba néz a nyíl.	<code>std::cout <<</code> és <code>std::cin >></code>
A program elindul, de nem történik semmi	A program adatra vár.	Írd be a kért adatot, és nyomj Entert.
Értelmetlen, furcsa szám jelenik meg	A változónak nincs értéke, vagy rossz adatot írtál be.	Adtál neki kezdőértéket, vagy megvolt a beolvasás?
Az eredményből hiányoznak a tizedesek	Két egész számot osztottál.	Az egyik szám legyen tizedes tört (<code>2.0</code>), vagy a változó típusa <code>double</code> .
A kiírt szövegek egybefolynak	Hiányzik a szóköz vagy a sorvége.	Tegyél szóközt az idézőjelen belülre, illetve <code>std::endl</code> -t a sor végére.

Összefoglaló táblázat (puska)

A váz, amely minden programba kell:

```
#include <iostream>

int main() {
    // ide jönnek az utasításaid
    return 0;
}
```

Minta	Jelentés
<code>std::cout << "szoveg";</code>	szöveg kiírása
<code>std::cout << 3 + 4;</code>	számolás, az eredmény kiírása
<code>std::cout << std::endl;</code>	új sor
<code>std::cout << "Kor: " << kor << std::endl;</code>	szöveg és változó együtt
<code>// megjegyzes</code>	megjegyzés, a gép nem veszi figyelembe
<code>int kor = 16;</code>	egész számot tároló változó, kezdőértékkel
<code>double atlag = 4.5;</code>	tizedes törtet tároló változó
<code>std::string nev = "Anna";</code>	szöveget tároló változó
<code>kor = kor + 1;</code>	a változó értékének megváltoztatása
<code>std::cin >> kor;</code>	beolvasás a billentyűzetről a változóba
<code>+, -, *, /, %</code>	összeadás, kivonás, szorzás, osztás, maradék

Figyelem! Három dolog, amit soha ne felejts el: minden utasítás végére pontosvessző kell; a szöveget idézőjelek közé írjuk, a változó nevét nem; két egész szám osztásakor a tizedesek elvesznek.